

Registro de actividades realizadas

Formato: [Actividad] - [Fecha] - [Integrantes] - [Duración] - [Commit] - [Descripción]

Sprint Planning y primeros avances: 15/04/2026 - Atrio, Robayna - 2h - 59d7656 - Planificación de la primera entrega, creación del repositorio con su tablero Kanban correspondiente y los labels para las issues. Se crearon todas definiciones necesarias para el marco de trabajo así como los roles y ceremonias. También se realizó la configuración del repositorio de Github con la estrategia de branching a utilizar.

Definición de proceso de ingeniería: 19/04/2026 - Viera - 45min - [commit-hash] - Análisis de la rúbrica y del trabajo previo del equipo para definir el primer proceso de ingeniería del proyecto: flujo de issues, nomenclatura de ramas y commits, estándares de código, gates de revisión por Pull Request, rotación de roles, herramientas y plan de evolución del proceso para las entregas siguientes.

Modelo de calidad, deuda técnica, revisión de docs y búsqueda de issues: 19/04/2026 - Atrio - 3h - [commit-hash] - Creación y revisión de modelo de calidad, deuda técnica, revisión de docs y búsqueda de issues.

Evidencia de resultados de la entrega

Definición de marco de gestión basado en Kanban

Se creó un tablero Kanban en Github projects con columnas: ToDo, InProgress, InReview, Done. Se le asignó a las columnas InProgress e InReview un WIP de 3 debido a que somos 3 desarrolladores.

<https://github.com/orgs/IngSoft-ISA2-2026-1/projects/31>

Roles y ceremonias: Se asignaron los siguientes roles: PO - Xoan Atrio, Encargado de ceremonias (similar a Scrum Master) - Lautaro Robayna, Desarrollo y Testeo - los 3. Los roles rotarán en cada iteración.

Se pensaron las siguientes ceremonias por iteración: Sprint planning al inicio(1h), una daily en la mitad(10 min), una review que sería una reunión con el PO(30 min) y una retrospective al finalizar(1h) que se hará con metro retro.

Definition of Ready: Escritura de US (como, quiero, para), criterios de aceptación, tiempo estimado y escenarios de prueba BDD, todo hecho y/o revisado por al menos 2 integrantes.

Definition of Done: Una feature será marcada como finalizada cuando pase todos los tests y sea revisada por el PO de esa iteración.

Definición de issues: Una issue será cualquier tipo de error o mejora que se pueda realizar sobre el proyecto actual y deberán tener labels asignados entre los siguientes:

feature, bug, docs, infra, telemetry, refactor. También deberán tener un título, una descripción. Los labels se pueden ver configurados en el siguiente link:

<https://github.com/IngSoft-ISA2-2026-1/proyecto-m7b-atrío-robayna-viera/labels>

Definición de un primer proceso de ingeniería

Se define el siguiente proceso de ingeniería inicial para la primera entrega. El mismo será revisado y ajustado en cada entrega posterior según corresponda (por ejemplo, incorporación de TDD en la Entrega 2 y BDD en la Entrega 3).

Flujo de trabajo: Toda funcionalidad nueva, bug, tarea de infraestructura o documentación se crea como issue en GitHub Projects con título, descripción, label (feature, bug, docs, infra o telemetry) y responsable asignado. Al crearse se ubica en ToDo. El integrante que toma un issue lo mueve a InProgress (respetando el WIP de 3), crea una rama desde main con la nomenclatura tipo/numero-descripcion (ej: feature/12-login-usuarios) y desarrolla el cambio. Al finalizar abre un Pull Request hacia main que referencia al issue (Closes #N) y lo mueve a InReview. El PR debe pasar los checks automáticos del pipeline (build, tests, linter) y ser aprobado por al menos otro integrante. Tras el merge, la rama se elimina, el issue pasa a Done y se cierra. Al cierre de la etapa se agrega el tag Entrega N sobre main.

Estándares de commits: Los mensajes siguen el formato tipo(alcance): descripción (ej: feat(auth): agrega endpoint login). A partir de la Entrega 2 se suman los rótulos requeridos por la rúbrica: [Green], [Refactor], [TDD-GREEN], [TDD-REFACTOR], [BDD-GREEN], [BDD-REFACTOR].

Estándares de código:

- Se aplica linter y formateador en ambos lados de la aplicación (ESLint + Prettier en Angular, analizadores de .NET en el back). No se mergea código que no pase el linter.
- Se respetan los criterios de clean code: nombres significativos, sin magic numbers ni strings, funciones cortas con responsabilidad única.
- Todo cambio en el backend debe estar respaldado por pruebas unitarias. El objetivo de cobertura es 100% según el modelo de calidad definido.
- No se versionan secretos ni datos sensibles. Las configuraciones se manejan por variables de entorno o archivos ignorados por git.

Revisión y gates: Todo cambio a main pasa obligatoriamente por Pull Request. El PR debe tener al menos un reviewer distinto del autor y debe pasar todos los checks automáticos del pipeline de CI (GitHub Actions) antes de poder mergearse.

Roles durante el proceso: Los tres integrantes actúan como desarrollador y tester en todas las entregas. El Product Owner de la iteración valida que el trabajo cumpla con los criterios de aceptación antes de que un issue pase a Done. El Encargado de ceremonias

garantiza que se estén cumpliendo las prácticas definidas (Definition of Ready, Definition of Done, revisión por pares). Los roles de PO y Encargado de ceremonias rotan en cada entrega.

Herramientas: Gestión de issues y tablero Kanban en GitHub Projects. Versionado y revisión en GitHub. CI/CD en GitHub Actions (se configura en detalle en la Entrega 2). Testing unitario con el framework de .NET en el backend y Jasmine/Karma en el frontend; Cypress + Cucumber para BDD a partir de la Entrega 3. Telemetría con Prometheus, OTLP y Grafana a partir de la Entrega 4.

Evolución del proceso: Este proceso es un punto de partida y se irá adaptando en las siguientes entregas. En la Entrega 2 se incorpora el pipeline CI/CD completo y TDD en el backend para la reparación de bugs. En la Entrega 3 se incorpora BDD colaborativo (PO, tester y desarrollador) para las nuevas funcionalidades, implementándolas primero en el front con BDD y luego en el back con TDD. En la Entrega 4 se incorpora telemetría, un nuevo tipo de testing y métricas DevOps para medir la performance del proceso mismo. En cada informe de avance se reportará si hubo cambios al proceso de ingeniería o si se mantiene estable.

Configuración del repositorio en GitHub

Se decidió usar una metodología Trunk based para un desarrollo más rápido. Siguiendo el proceso de ingeniería cuando se vaya a realizar un cambio de cualquier tipo cada integrante creará un issue y lo añadirá al GitHub Projects (en caso de no estar creado previamente), y se creará una rama independiente para este cambio. Cuando el cambio se haya realizado con éxito se realizará la Pull Request y posteriormente se mergeará a main. Idealmente todas las ramas se mergearán a main el mismo día en el que fueron creadas con el fin de acercarse a una metodología similar a Trunk, aunque pueden habilitarse excepciones a esto si el equipo lo ve necesario.

En cuanto a la organización de archivos y directorios en la raíz se encuentran las carpetas principales que separan responsabilidades y secciones, de entre estas la principal es Material, la cual cuenta con todo el código fuente de la aplicación, así como todo el material adicional que fue brindado sobre la misma como la letra, documentación original, scripts de la base de datos y exportaciones (json). El código fuente tiene una clara separación entre el backend y el frontend, donde el backend se separa por capas y tests para las mismas con carpetas adicionales como Instrumentation. El frontend en cambio

También se tiene la carpeta Documentación en la que se encuentra el informe de cada entrega. Siguiendo este principio se añadirán carpetas conforme sea necesario para añadir nuevos aspectos del proyecto.

Para los estándares de nomenclatura se utilizarán los propuestos en el proceso de ingeniería, siendo estos **tipo/numero-descripción** para los nombres de ramas, **tipo(alcance): descripción** para los commits, **Entrega N** para los tags donde “N” será el número de la entrega y **Closes #N** para Pull Requests (puede haber excepciones si requiere Pull Requests específicas).

Análisis de deuda técnica y gestión de calidad

En este proyecto se definió un modelo de calidad basado en los atributos de calidad que pueden ser priorizados en un proyecto universitario.

Mantenibilidad

- Que cumpla con todos los criterios de clean code relevantes (desde magic numbers o strings hasta comentarios y formato)
- Cumplimiento de la guía de estilos utilizada con formateador automático
- Verificación de complejidad de métodos, sin métodos más complejos de lo necesario. Tanto en escala $O(n)$ como en tamaño y complejidad de entendimiento.

Usabilidad

- Todas las acciones principales deben poder realizarse con 3 clicks desde el menú principal.
- La estética de la página debe ser consistente y accesible, verificar con wave.

Robustez ante errores

- Porcentaje de entradas inválidas que el sistema maneja sin romperse debe de ser del 100% (porcentaje de excepciones sobre total de caminos inválidos).

Portabilidad

- La aplicación debe ser 100% portable en sistemas virtuales como docker
- Uso correcto de archivos de configuración y variables de entorno
- Se debe poder ejecutar el sistema solo con descargarlo y ejecutar un comando.

Testeabilidad

- Se debe tener un pipeline CI/CD para tests
- 100% de coverage en todos modulos
- Cubrir todos los caminos de éxito y todos aquellos caminos con fallos que sean relevantes.
- Pruebas unitarias con cobertura total más pruebas de integración requeridas.

Desplegabilidad

- Se debe tener un pipeline CI/CD para el despliegue, totalmente automatizado.
- Desde un push a main se debe tener un despliegue en menos de 5 minutos.
- Se debe contar con versionado con posibilidad de rollback.

Seguridad

- El código debe cumplir con medidas de seguridad básica (integrar análisis de vulnerabilidades en el código automatizado)
- Principalmente cumplir con no guardar datos sensibles en bruto ni retornar más información de la necesaria. Verificar permisos.

Observabilidad y Monitoreo

- Se cuenta con logs estructurados para todas las funcionalidades principales.
- Se utilizan herramientas para monitorear el estado del sistema en tiempo real.
- Se envían alertas automatizadas bajo ciertas condiciones del sistema.

Performance

- Tiempos de respuesta menores a 1 segundo en tráfico esperado (menos de 100 usuarios).

Para el análisis de deuda técnica se analizó el proyecto presentado en base al modelo de calidad y conocimientos previos de codificación, esta se evaluará en base a issues creadas en el repositorio donde cada una representa un incumplimiento o una “deuda técnica” a resolver. En base a la evaluación del proyecto podemos concluir que aunque existan problemas de clean code, tests, versiones antiguas y otros, la principal deuda técnica se encuentra en la seguridad del sistema (Ej: guardar contraseña en texto plano) y en la falta de CI/CD que puede ser proporcionado mediante pipelines para tests y despliegue.

Mejoras en la calidad y evidencia de revisión con el PO

Con base en todos los estándares y procesos planteados se espera que al resolver los problemas detectados y se continúe con un proceso de mejora continua se vean resultados directos en la calidad del proyecto tanto desde el punto de vista del cliente como de los desarrolladores del mismo. En esta entrega no se realizaron cambios directos a la aplicación por lo que la revisión con el PO fue dirigida hacia la documentación y esta primera etapa de planificación, se tuvieron en cuenta errores sencillos como información duplicada en varias secciones. Pero el enfoque de esta revisión fue principalmente sobre el primer proceso de ingeniería y el modelo de calidad diseñado, postergando aspectos como “Performance”, definiendo métricas más precisas como el porcentaje de entradas inválidas como excepciones y clarificando estándares de código.

Retrospectiva y acciones de mejora

le